

Scaling AI

Data Parallelism and Distributed Training

Compute • Communication • Scaling efficiency • Bottleneck diagnosis

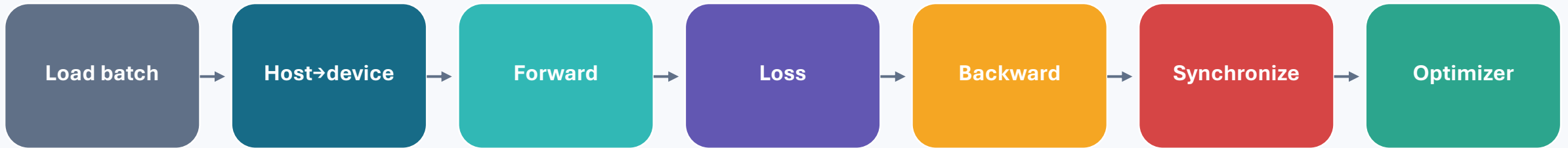
01

What does one training step cost?

Build a time and memory model before adding GPUs.

Anatomy of one training step

The critical path contains more than GPU math.



$$T_{\text{step}} = T_{\text{input}} + T_{\text{transfer}} + T_{\text{forward}} + T_{\text{backward}} + T_{\text{communication}} + T_{\text{optimizer}} + T_{\text{overhead}}$$

THINK Which terms shrink when the batch is split across more GPUs?

Training memory per replica

Data parallelism replicates model states on every worker.

Parameters

2N bytes in BF16

Gradients

often about 2N bytes

Optimizer

Adam moments can dominate

Activations

batch, sequence, and layer dependent

Buffers

kernels, buckets, fragmentation

THINK Adding data-parallel GPUs does not reduce these replica costs.

Worked example: compute per worker

A larger local batch gives more computation to hide communication.

Case A

Global batch 256
8 GPUs → local batch 32

Case B

Global batch 256
32 GPUs → local batch 8

Same gradients

Model-gradient size does not shrink with local batch.

compute / communication ratio usually drops as local batch shrinks

Likely result

32 GPUs may scale worse.

Possible mitigation

larger global batch or accumulation

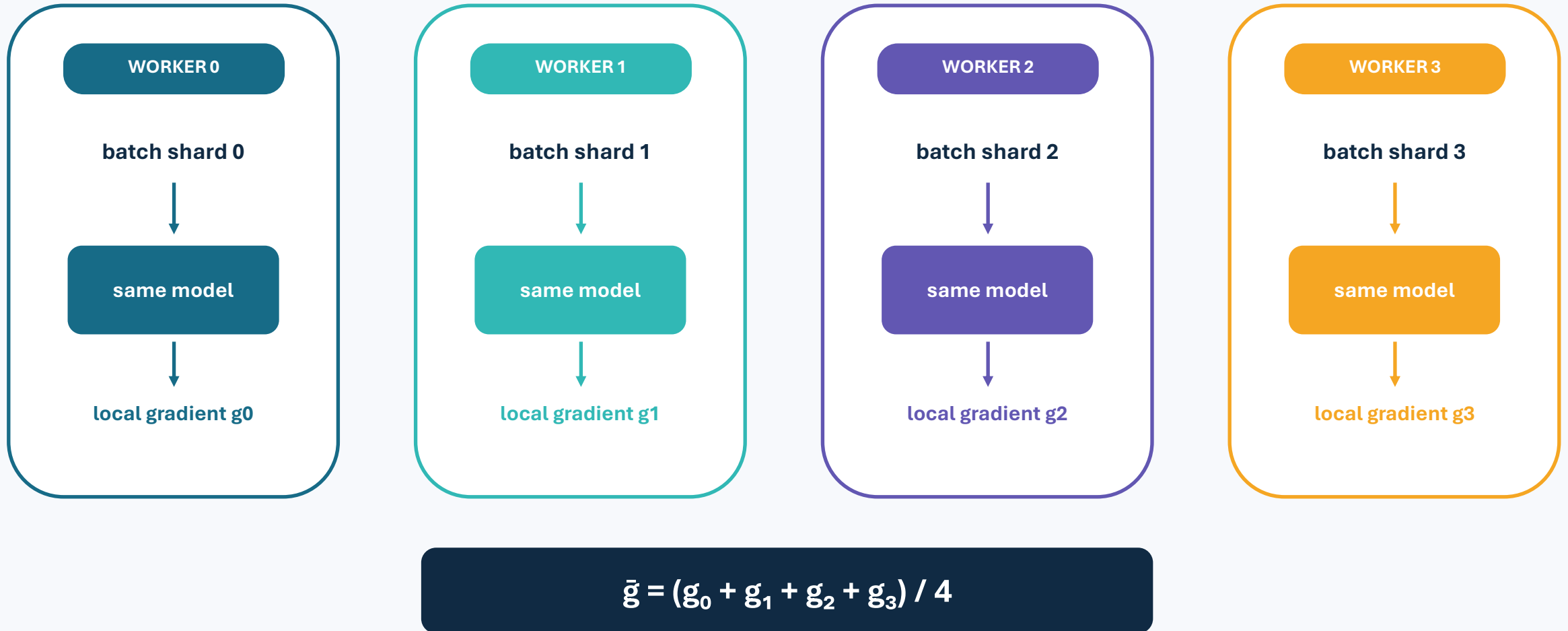
02

Synchronous data parallelism

Replicate the model, split the batch, average gradients.

Data-parallel training diagram

Every worker starts equal and applies the same averaged update.



THINK Why do all replicas stay identical after the optimizer step?

Worked example: why averaging gradients works

Split a batch of four examples across two workers.

Worker 0

example gradients: 2 and 4
local mean $g_0 = 3$

Worker 1

example gradients: 6 and 8
local mean $g_1 = 7$

AllReduce average

$$\bar{g} = (3 + 7) / 2 = 5$$

$$\text{Single global batch mean} = (2 + 4 + 6 + 8) / 4 = 5$$

Condition

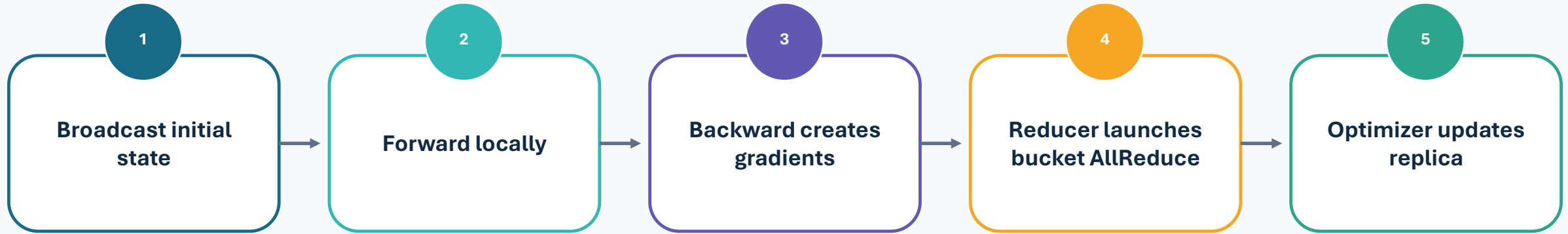
equal local batch sizes

Watch out

unequal shards need weighting

What PyTorch DDP does under the hood

Gradient hooks trigger bucketed synchronization during backward.



Why buckets?

Amortize overhead and overlap communication with backward.

Why hooks?

Launch synchronization as gradients become ready.

Minimal DDP control flow

One process per GPU; one model replica per process.

```
dist.init_process_group(backend="nccl")
torch.cuda.set_device(rank)
model = Model().to(rank)
ddp = DDP(model, device_ids=[rank])

for x, y in loader:
    optimizer.zero_grad()
    loss = criterion(ddp(x), y)
    loss.backward()      # sync hooks fire
    optimizer.step()
```

Process group

defines participating ranks

Distributed sampler

gives each rank different examples

Synchronization

happens during backward

Global batch size is a systems and optimization choice

Scaling throughput changes the optimization regime.

$$B_{\text{global}} = B_{\text{per-GPU}} \times \text{DP} \times \text{accumulation steps}$$

Example A

32 per GPU $\times$ 8 GPUs $\times$ 1
= 256

Example B

8 per GPU $\times$ 8 GPUs $\times$ 4
= 256

Difference

B synchronizations differ unless no_sync is used.

THINK Same global batch does not guarantee the same wall-clock behavior.

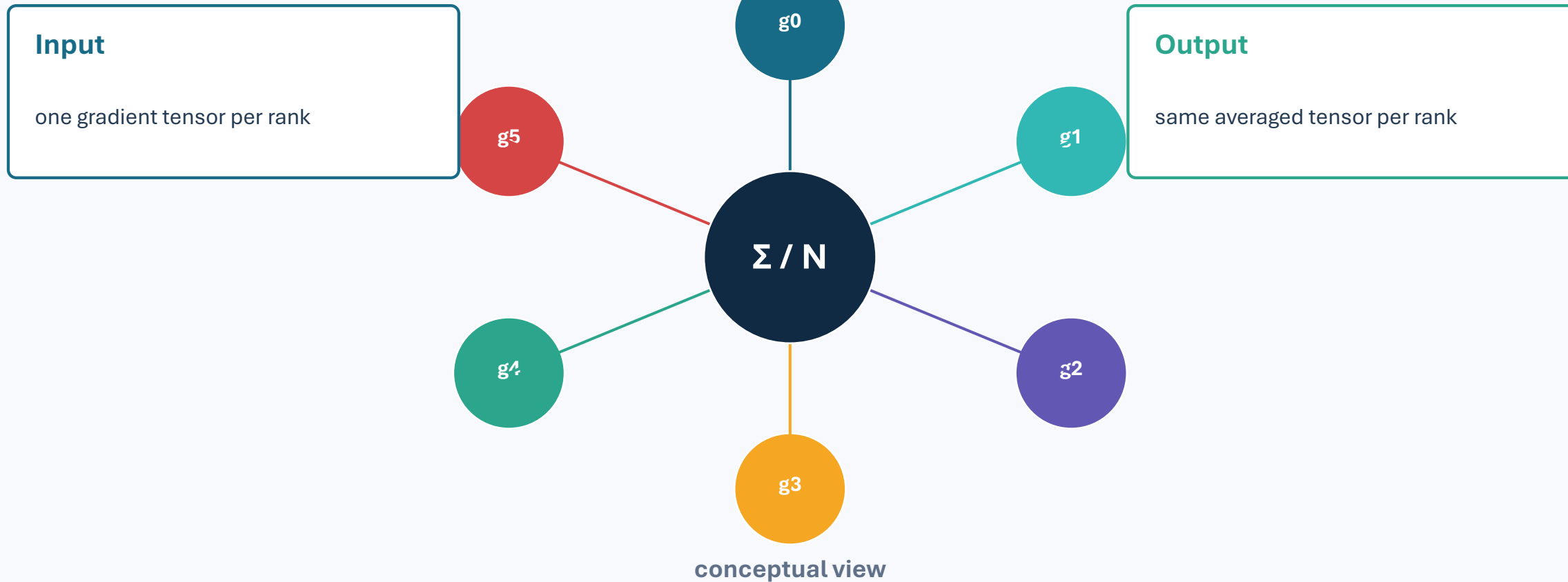
03

Collective communication

How gradients become identical across workers.

AllReduce: reduce, then distribute

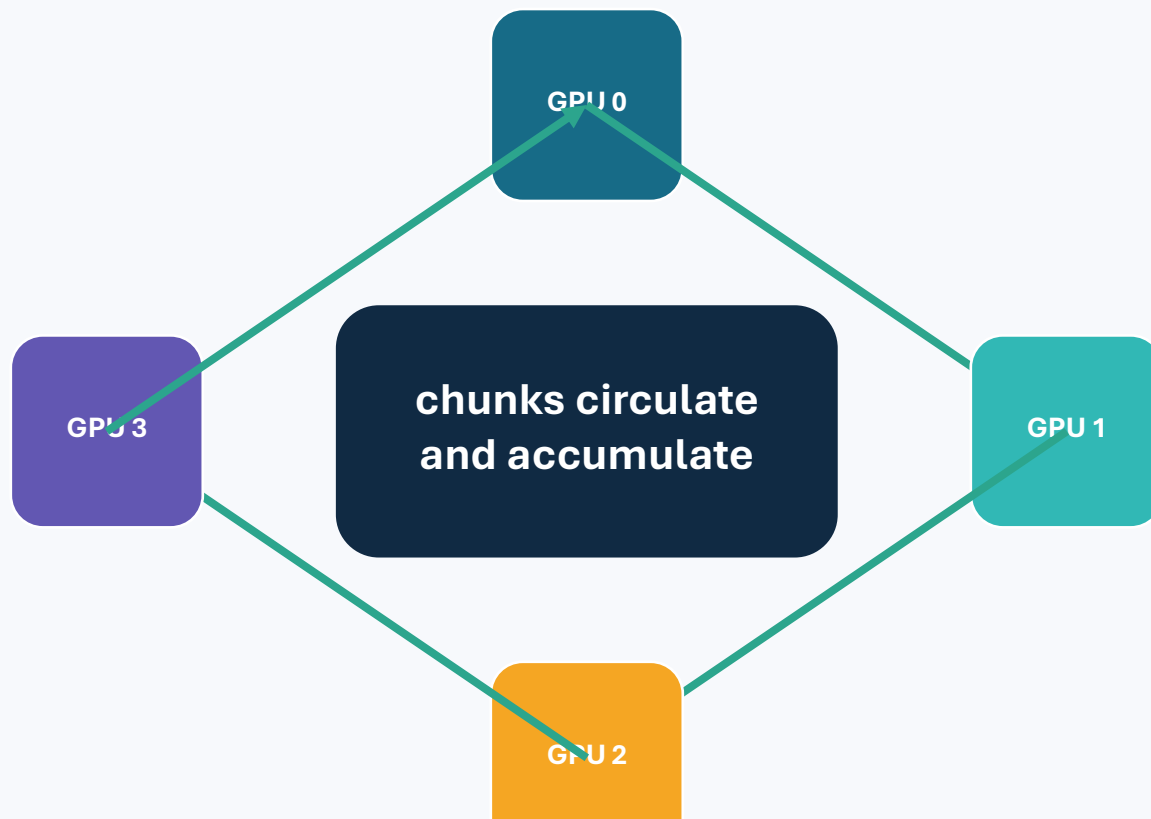
Every worker contributes a tensor and receives the final reduction.



THINK A collective is logically global even when implemented without a central server.

Ring AllReduce intuition

ReduceScatter followed by AllGather.



Phase 1

ReduceScatter: reduce each chunk

Phase 2

AllGather: distribute reduced chunks

Benefit

balanced large-message bandwidth

A simple communication-time model

Latency dominates small messages; bandwidth dominates large messages.

$$T_{\text{message}} \approx \text{latency} + \text{message size} / \text{bandwidth}$$

Small tensor

1 KB / 25 GB/s is tiny
but launch latency remains

Large tensor

1 GB / 25 GB/s $\approx$ 40 ms
bandwidth dominates

Why buckets?

combine many small gradients
into fewer large collectives

THINK Why are many tiny AllReduce calls usually inefficient?

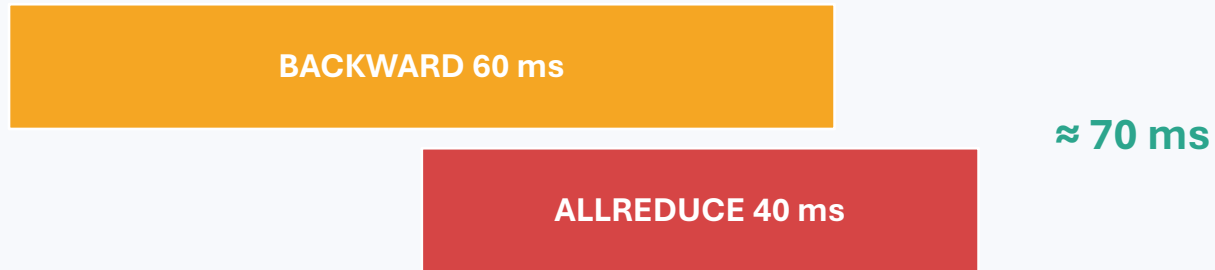
Overlap communication with backward computation

Expose only the communication that remains on the critical path.

No overlap



Overlap

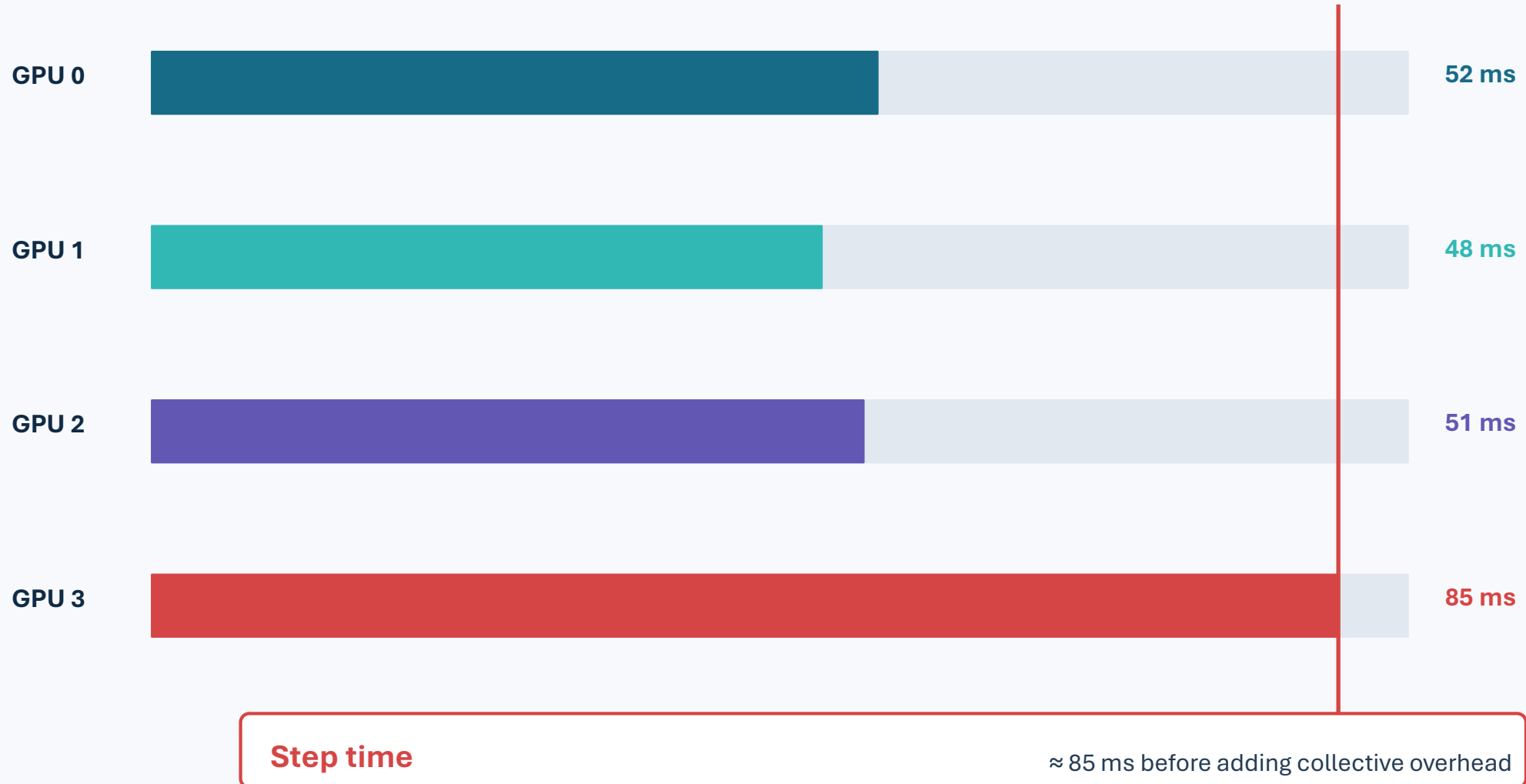


Worked result

30 ms of communication is hidden; 10 ms remains exposed.

Synchronization makes the slowest worker visible

A barrier-like collective waits for every participant.



04

Scaling efficiency

Measure speedup, not GPU count.

Speedup and efficiency

Ideal scaling is a reference line, not an expectation.

$$\text{Speedup}(N) = T_1 / T_n$$

$$\text{Efficiency}(N) = \text{Speedup} / N$$

Measured

$T_1 = 100$ s
 $T_4 = 30$ s

Speedup

$100 / 30 = 3.33\times$

Efficiency

$3.33 / 4 = 83.3\%$

THINK Where did the missing 16.7% go?

Strong scaling vs weak scaling

The question changes when the workload changes.

STRONG SCALING

fixed global batch



more GPUs → less work per GPU

Goal: finish the same job faster

WEAK SCALING

fixed local batch



more GPUs → larger global batch

Goal: maintain time for a larger job

Worked example: Amdahl's law

Even ideal parallel work is limited by the serial fraction.

$$S(N) = 1 / ((1 - P) + P/N)$$

Given

P = 0.95 parallel
N = 8 GPUs

Speedup

$1 / (0.05 + 0.95/8)$
= 5.93×

Efficiency

$5.93 / 8$
= 74.1%

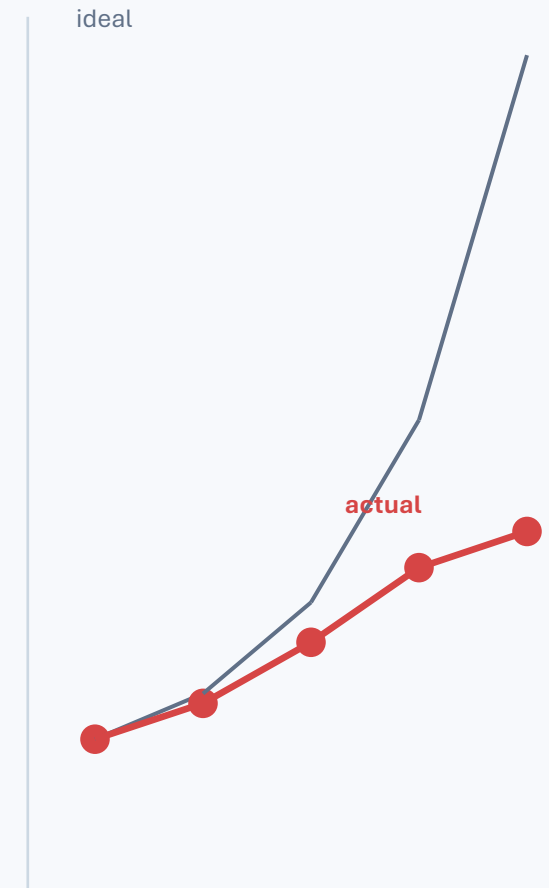
Infinite-GPU limit: $1 / 0.05 = 20\times$

THINK What part of training behaves like a serial or synchronization fraction?

Worked scaling experiment

Runtime improves, but efficiency falls as communication dominates.

GPUs	Time (s)	Speedup	Efficiency
1	100	1.00×	100.0%
2	56	1.79×	89.3%
4	32	3.13×	78.1%
8	21	4.76×	59.5%
16	18	5.56×	34.7%



THINK At what GPU count is adding another GPU barely worthwhile?

Bottleneck diagnosis matrix

Use symptoms to choose the next experiment.

Low GPU utilization

input stalls or tiny local batch

profile loader; increase workers or batch

Long collective time

network or oversized gradient traffic

measure bandwidth; overlap; compress/bucket

One slow rank

straggler or imbalance

per-rank timing; inspect data and hardware

High GPU use, poor scaling

communication hidden poorly

timeline trace; tune bucket size

Loss changes with GPU count

global batch or sampling changed

verify sampler, reduction, LR, seeds

THINK Never optimize before identifying the dominant term on the critical path.

Common data-parallel pitfalls

Correct execution is not guaranteed by “the code ran.”

Duplicate samples

sampler not sharded or epoch seed not updated

Unequal work

last batch, variable sequence length, preprocessing

Wrong reduction

sum vs mean changes effective gradient scale

Too-small local batch

kernel inefficiency and exposed communication

Oversubscription

too many CPU workers or shared GPU device

Checkpoint mismatch

only one rank saves, but all must restore consistently

Five ideas to remember

- 1 DDP replicates the model and splits the data.
- 2 Gradient synchronization preserves equivalent updates.
- 3 Communication cost depends on latency, bandwidth, and gradient size.
- 4 Strong scaling eventually shrinks local work too far.
- 5 **Measure speedup, efficiency, and the critical-path bottleneck.**